Advanced Core-Local Interrupt Controller (ACLIC) RISC-V Privileged Architecture Extensions

Version v0.16, 2025-09-15: Draft

Table of Contents

Preamble ... 1
Copyright and license information ... 2
1. Introduction ... 3
2. Advanced Interrupt Architecture for Embedded Systems (Smaiae) ... 5
3. Advanced Core Local Interrupt Controller (Smaclic and Ssaclic) ... 6
3.1. New Interrupt Delivery Mode ... 6
3.2. Indirect CSR access to APLIC registers ... 7
3.2.1. Interrupt Priority (iprio) ... 7
3.2.2. Source Configuration (sourcecfg) ... 7
3.3. APLIC configured for ACLIC operation ... 8
3.4. State Enable ... 9
4. Conditional Stack Pointer Swap extension (Smcsps, Sscsps) ... 10
4.1. New stack pointer for conditional swap (`xsp`) CSR ... 10
4.2. Instructions for conditional stack pointer swap ... 10
4.3. State Enable ... 11
4.4. Instructions ... 12
4.4.1. cspspush ... 12
4.4.2. cspspop ... 13
5. Horizontal Nested Interrupt Preemption Support (Smnip & Ssip) ... 14
5.1. Changed and new CSRs at Machine Level ... 14
5.1.1. New Interrupt Preemption Configuration (`mipreemptcfg`) CSR ... 14
5.1.2. New Interrupt enable threshold (`mithreshold`) CSR ... 14
5.1.3. New Previous Interrupt Status (`mpistatus`) CSR ... 14
5.2. Changed and new CSRs at Supervisor Level ... 15
5.2.1. New Previous Interrupt Status (`spistatus`) CSR ... 15
5.2.2. New Interrupt enable threshold (`sithreshold`) CSR ... 15
5.3. Reset Behavior ... 15
5.3.1. Mandatory reset state ... 15
5.4. Operation ... 16
5.4.1. Trap behavior ... 16
5.4.2. Returns from Handlers ... 16
5.5. State Enable ... 16
6. Support for interrupt vector table (Smivt, Ssivt) ... 17
6.1. Changed and new CSRs ... 17
6.1.1. New `xivt` and `xeivt` CSR ... 17
6.1.2. New `xtvec` CSR Mode ... 17
6.1.3. Smivt Changes to `xtvec` CSR Mode ... 17
6.1.4. Vector table fault handling ... 18
6.1.5. State Enable ... 19
7. Synchronous exceptions hardware vectoring- Smehv ... 20
8. Trap handler push and pop extension (Smtp) ... 21
8.1. Trap handler push and pop functional overview ... 21
8.2. Late Preemption of the interrupt handler ... 22

8.3. Tail chaining of interrupts handlers 22
8.4. Stack layout 23
8.5. Fault handling 24
8.6. Debug 24
8.7. Instructions 25
8.7.1. tpush 25
8.7.2. tpopxret 27

Preamble

Graphics used are either explicitly available for free, are property of RISC-V International, or were created using Wavedrom.

Copyright and license information

This RISC-V ACLIC extension specification is © 2018-2025 RISC-V international

This document is released under a Creative Commons Attribution 4.0 International License. creativecommons.org/licenses/by/4.0/.

Chapter 1. Introduction

The set of extensions defined in this specification enhance the existing interrupt specifications. The Advanced Core Local Interrupt Controller (ACLIC) is a single hart interrupt controller defined for:

- Low latency interrupt handling (achieved through nested preemption)
- High throughput interrupt handling
- Up to 1023 external interrupts
- Low area overhead
- Maximize compatibility with AIA

The table below provides a summary of the ACLIC extensions.

Extension Name	Description
Smaiae	Advanced Interrupt Architecture for Embedded Systems
Smaclic	Advanced Core Local Interrupt Controller at Machine level
Ssaclic	Advanced Core Local Interrupt Controller at Supervisor level
Smivt	Support for interrupt vector table at Machine level
Ssivt	Support for interrupt vector table at Supervisor level
Smehv	Synchronous exceptions hardware vectoring
Smcsps	Conditional stack pointer swap at Machine level
Sscsps	Conditional stack pointer swap at Supervisor level
Smnip	Horizontal Nested Interrupt Preemption support at Machine level
Ssnip	Horizontal Nested Interrupt Preemption support at Supervisor level
Smtp	Support for trap handler push/pop

As this specification aims for single-hart systems, no support for the H extension is foreseen. Future specifications could extend the scope of these extension to also be applicable in systems with H extension.

Some extension depend upon other extensions. This table summarizes all dependencies. Each row contains the dependencies of the extension named in the first column.

Extension Name	Sm	Ss	Smcsrind	Sscsrind	Smaiae	Smnip
Smivt	x					
Ssivt		x				
Smcsps	x					
Sscsps		x				
Smtp	(x)	(x)				
Smaiae	x	(x)	x	(x)		
Smaclic					x	
Ssaclic					x	
Smnip					x	
Ssnip						x
Smehv						x

In this table (x) denotes a dependency that is valid, if the corresponding privilege mode is implemented in the hart.

The extensions defined here are orthogonal to the NMI and RNMI mechanisms. Their behavior is unchanged by the extensions of ACLIC.

Chapter 2. Advanced Interrupt Architecture for Embedded Systems (Smaiae)

The Smaia extension encompasses all added CSRs and all modifications to interrupt response behavior that the AIA specifies for a hart, over all privilege levels.

The Smaiae extension is a subset of Smaia. To limit the implementation cost, the interrupt filtering and virtual interrupts are not supported.

Specifically, `xvip` and `xvien` are read-only 0. When XLEN=32, `xviph` and `xvienh` are also read-only 0.

While other features of AIA like IMSIC or programmable IPRIO might also not be needed resource constraint systems, These are already controlled by WARL registers, so implementations can already choose to not support these.

Chapter 3. Advanced Core Local Interrupt Controller (Smaclic and Ssaclic)

The Smaclic and Ssaclic extensions depend on the Smaiae extension.

Of the main ACLIC goals, these targets are addressed here:

- Up to 1023 external interrupts
- Low area overhead
- Maximize compatibility with AIA

To achieve this, ACLIC builds upon the hart-level extensions of AIA, and the Advanced Platform Level Interrupt Controller (APLIC). The ACLIC is a tailored APLIC, optimized for resource efficiency, directly integrated with one hart.

In terms of compatibility, the goal is that a hart with Smaclic/Ssaclic extensions can be used at with the same trap handlers as one developed for AIA.

At config-time, the interface differs, but ACLIC uses the same structure for configuration as the APLIC.

3.1. New Interrupt Delivery Mode

The Incoming Message Signaled Interrupt Controller (IMSIC) contains the `eidelivery` CSR, which defines the interrupt delivery mode. A new delivery mode is defined:

0 =	Interrupt delivery is disabled
1 =	Interrupt delivery from the interrupt file is enabled
0x20000000 =	Interrupt delivery from an ACLIC (new)
0x40000000 =	Interrupt delivery from a PLIC or APLIC is enabled

If Smaclic or Ssaclic extensions are present, reset initializes `eidelivery` to 0x20000000.

When interrupts are delivered from an ACLIC, the following behavior is valid:

The IMSIC registers `eithreshold`, `eip`*k* and `eie`*k* serve the same functionality as with `eidelivery` = 1.

The `eip` array and `eie` array act as an alias of the pending and enable bits of the connected APLIC domain.

The `eithreshold` CSR is an alias of the `ithreshold`. For the interrupt enable and interrupt pending bits `eip`*k* and `eie`*k*, a write with the value 1 performs a write access with the value 1 to the corresponding `setip` or `setie` register, while a write with the value 0 performs a write access with value 1 to the corresponding `in_clrip` or `clrie` register. On read of `eip`*k* or `eie`*k*, these CSRs are an alias of `setip`[*k*] and `setie`[*k*], respectively.

For XLEN=32, `eip`*k* and `eie`*k* access `setip`[*k*]/`in_clrip`[*k*] and `setie`[*k*]/`clrie`[*k*], respectively. For XLEN=64, `eip`*k* and `eie`*k* only exist for even numbered indices of *k*. In this case, `eip`*k* and `eie`*k* access `setip`[*k*]/`in_clrip`[*k*] and `setie`[*k*]/`clrie`[*k*] in the lower half of the CSR, and `setip`[*k+1*]/`in_clrip`[*k*] and `setie`[*k+1*]/`clrie`[*k*] in the upper half of the CSR, respectively.

*Mirroring these APLIC registers at a core level has several advantages. * It removes unnecessary resource duplication between IMSIC and APLIC when integrated with a single hart. * It simplifies the address decoding logic within the interrupt controller by using CSR*

> *access over memory mapped ones. * It simplifies the handling of level-sensitive interrupts, avoiding additional checking of the pending flag in the APLIC in a handler of a level sensitive interrupt. * It provides a unified interface for all harts from a SW point of view, without the need to know a platform defined start address. * It simplifies the register interface, as CSR instructions have single bit operations*

The `xtopei` registers work analogous to the IMSIC operation, but map to the current highest-priority pending-and-enabled interrupt of the connected APLIC domain. In this delivery mode, the `xtopei` CSR reflect both the interrupt identity and the interrupt priority.

bits 26:16	Interrupt identity
bits 10:0	Interrupt priority (configured target[identity].iprio)

3.2. Indirect CSR access to APLIC registers

The APLIC registers used in the ACLIC configuration can all be accessed by indirect CSRs, eliminating the overheads associated with a memory mapped device, and presenting a consistent SW interface for any hart.

To have full control over the necessary APLIC registers, the following indirect CSR access is added.

3.2.1. Interrupt Priority (iprio)

When XLEN = 32, each `xireg` register controls the ACLIC priority setting of four interrupts.

`xiselect`	`xireg` size	`xireg` state
0x1000	4B	`xacliciprio0`
…	…	…
0x1100	4B	`xacliciprio255`

Indirect access to `xacliciprio[k]` mirror `target[k*4].iprio` up to `target[k*4+3].iprio`.

When XLEN = 64, only the even-numbered registers exist and each register controls the priority setting of eight interrupts. Indirect access to `xacliciprio[k]` mirror `target[k*4].iprio` up to `target[k*4+7].iprio`.

3.2.2. Source Configuration (sourcecfg)

When accessing the source configuration via indirect CSR, only the lower 16b are accessible, as the rest of the register is reserved.

When XLEN = 32, the `xireg2` and `xireg3` registers combined control the source configuration of four interrupts.

`xiselect`	`xireg2/3` size	`xireg2` state	`xireg3` state
0x1000	4B	`xaclicsourcecfg0`	`xaclicsourcecfg1`
…	…	…	…
0x1100	4B	`xaclicsourcecfg510`	`xaclicsourcecfg511`

Indirect access to `xaclicsourcecfg[k]` mirrors `sourcecfg[k*2][15:0]` up to `sourcecfg[k*2+1][15:0]`.

When XLEN = 64, only the even-numbered registers exist and the `xireg2` and `xireg3` registers combined

control the source configuration of eight interrupts. Indirect access to `xaclicsourcecfg[k]` mirrors `sourcecfg[k*4][15:0]` up to `sourcecfg[k*4+3][15:0]`.

3.3. APLIC configured for ACLIC operation

The APLIC contains a per-domain configuration register, which, among other things, allows selecting the delivery mode. The `DM` field is extended to two bits, which allows encoding a new mode for ACLIC delivery.

0 =	direct delivery mode
1 =	MSI delivery mode
2 =	ACLIC delivery mode (new)

If Smaclic and/or Ssaclic are implemented, the reset value of the delivery mode for the respective domains shall be 'ACLIC delivery mode'.

The extended `domaincfg` register has this format:

bits 31:24	read-only 0x80
bit 8	IE
bit 7	read-only 0
bits 3:2	DM (WARL)
bit 0	BE (WARL)

All other register bits are reserved and read as zeros.

When in ACLIC delivery mode, the following, additional restrictions are applied:

bits 31:24	read-only 0x80
bit 8	IE = 1 (read-only)
bit 7	read-only 0
bits 3:2	DM = 2
bit 0	BE (WARL)

Specifically, = `IE` bit is fixed to 1, as its functionality is subsumed by the `xstatus.xIE` bits at the hart.

If Smaclic and/or Ssaclic are implemented, the reset value of the delivery mode for the respective domains shall be 'ACLIC delivery mode'. In ACLIC-only systems, access to this register is not needed, and therefore no indirect CSR access is provided.

In the ACLIC delivery mode, there the following registers are not accessible and not used:

- As the ACLIC is not MSI capable, xmsiaddrcfg[h] and genmsi registers are not required.
- Interrupt delivery control (IDC) structure is only needed in direct delivery mode, and not applicable to ACLIC delivery.
- The indirect CSR access simplifies setting and clearing of individual bits in the pending and enable arrays. Therefore, the registers setipnum, clripnum, setienum, clrienum, setipnum_be, and setipnum_le are not used in the ACLIC configuration.
- The target registers are not fully implemented. The iprio portion of it can be accessed as described in the section on indirect access. The rest is not required.

3.4. State Enable

Access to existing and repurposed AIA CSRs is already controlled by the state enables defined in Smaia/Ssaia.

Additional control is provided for the newly added registers `xacliciprio[k]` and `xaclicsourcecfg[k]`.

If the Smstateen extension is implemented, then the bit ?? in mstateen0 is implemented. If bit ?? of a controlling mstateen0 CSR is zero, then access to the new CSRs (`maclicipriо[k]`, `maclicsourcecfg[k]`, `saclicipriо[k]`, `saclicsourcecfg[k]`) by S-mode or a lower privilege mode results in an illegal instruction exception, except if the hypervisor extension is implemented, and the conditions for a virtual instruction exception apply, in which case a virtual instruction exception is raised when in VS or VU mode instead of an illegal instruction exception.

Chapter 4. Conditional Stack Pointer Swap extension (Smcsps, Sscsps)

The Sscsps depends on the Smcsps extension and adds the ability for conditional stack pointer swap at supervisor level and below.

For security and stack safety reasons, it can be desired to separate the stack areas per privilege level and between traps and normal execution. Performing stack pointer swapping in SW can be time consuming, as the conditions for swapping have to be checked first. However, stack pointer swapping is among the first things that need to happen even before saving the interrupted context. To ensure fast interrupt handling, this extension introduces an instruction to accelerate conditional stack pointer swapping.

4.1. New stack pointer for conditional swap (`xsp`) CSR

New CSRs are introduced to hold an alternative stack pointer. Conditionally, this stack pointer can be swapped with the currently active one.

```
      Number  Name        Description
(NEW) 0x348   msp         M-mode stack pointer for conditional swap (WARL)
(NEW) 0x148   ssp         S-mode stack pointer for conditional swap (WARL)
```

The CSR is defined as follows:

XLEN-1 … 4	3 … 2	1	0
SP[XLEN-1:4] (WARL)	WPRI	PUSH (WARL)	PPUSH (WARL)
XLEN-4	2	1	1

In addition to the stack pointer, this register holds two additional fields, `PUSH` and `PPUSH` (as in previous `PUSH`). These indicate, if a stack pointer swap is expected during a push or pop sequence, respectively.

> *As per the RISC-V calling convention, stack pointers are 128b aligned. Therefore the four least significant bits of the stack pointer are zero.*

4.2. Instructions for conditional stack pointer swap

To simplify and accelerate the conditional stack pointer swap, two instructions are introduced, Conditional stack pointer swap (push mode) and Conditional stack pointer swap (pop mode). To decide, if a stack pointer swap is required, these instructions evaluate the two LSBs of the register. The `PUSH` bit indicates that the stack pointer in `xsp` shall be swapped during the push sequence of the next handler.

The `PPUSH` bit indicates that the stack pointer shall be swapped back into `xsp`. It corresponds to the value of the `PUSH` bit in the previous context. This bit is part of the context of an interrupt handler, and needs to be saved and restored if a handler can be preempted.

Software can deactivate the stack pointer swap mechanism by writing zero to `xsp.PUSH`.

An example sequence with one vertical and one horizontal trap will look like this:

```
Mode  Event            msp.PUSH  msp.PPUSH  sp-swap
S     -                1         X          n
M     trap             1         X          n
```

```
M     cspspush         0         1         y
M     save context     0         1         n
M     trap             0         1         n
M     cspspush         0         0         n
M     cspspop          0         0         n
M     mret             0         0         n
M     restore context  0         1         n
M     cspspop          1         0         y
S     mret             1         0         n
```

The stack pointer is swapped when changing from S-mode to M-mode, and is swapped back when returning to S-mode. No change of stack pointer occurs for the cspspush and cspspop instruction in the nested handler.

4.3. State Enable

If the Smstateen extension is implemented, then the bit 53 (ACLIC) in mstateen0 is implemented. If bit 53 (ACLIC) of a controlling mstateen0 CSR is zero, then access to the new CSR `ssp` by S-mode or a lower privilege mode results in an illegal instruction exception, except if the hypervisor extension is implemented, and the conditions for a virtual instruction exception apply, in which case a virtual instruction exception is raised when in VS or VU mode instead of an illegal instruction exception.

4.4. Instructions

4.4.1. cspspush

Synopsis

Conditional stack pointer swap (push mode)

Mnemonic

cspspush rd, rs

Encoding

31–25	24–20	19–15	14–12	11–7	6–0
0000000	01001	00010	000	00010	1110011
		rs = x2	PRIV	rd = x2	SYSTEM

Description

This instruction conditionally swaps out the stack pointer and is typically used while pushing existing context. The behavior of the operations is:

```
xsp.PPUSH = xsp.PUSH          /* Latch push bit into the previous push bit */
if (xsp.PUSH == 1)            /* stack pointer shall be swapped */
    tmp = rs                  /* save current sp for writing later */
    rd = xsp.SP               /* Write the stack pointer with the low bits
cleared */
    xsp.SP = tmp[XLEN-1:4]    /* save original value of sp into xsp */
    xsp.PUSH = 0              /* Indicates the stack pointer has been swapped */
```

Included in: [smcsps]

4.4.2. cspspop

Synopsis

Conditional stack pointer swap (pop mode)

Mnemonic

cspspop

Encoding

31–25	24–20	19–15	14–12	11–7	6–0
0000000	01100	00010	000	00010	1110011
		rs = x2	PRIV	rd = x2	SYSTEM

Description

This instruction performs the opposite operation of cspspush, and is used to pop the previous context. A swap is performed, if the `PPUSH` field is set.

Behavior

```
xsp.PUSH = xsp.PPUSH          /* Restore PUSH bit from PPUSH */
if (xsp.PPUSH == 1)
    tmp = xsp.SP              /* save xsp.SP as temporary value */
    xsp.SP = rs[XLEN-1:4]     /* write current stack pointer to xsp */
    xsp.PPUSH = 0             /* Clear previous push bit */
    rd = tmp                  /* write original xsp value to stack pointer */
```

Included in: [smcsps]

Chapter 5. Horizontal Nested Interrupt Preemption Support (Smnip & Ssip)

The Smnip extension depends on the Smaiae extension and adds nested preemption support at machine level. The Ssnip extension depends on the Smnip extension and adds nested preemption support at supervisor level.

The extensions enable higher priority interrupts to preempt a lower priority interrupts in a nested fashion. It provides additional state to resume execution of the previous handler after a nested preemption.

5.1. Changed and new CSRs at Machine Level

```
       Number  Name         Description
 (NEW) 0x346   mpistatus    Previous interrupt context
 (NEW) 0xFB1   mithreshold  Interrupt enable threshold
 (NEW) 0x347   mipreemptcfg Interrupt preemption configuration
```

5.1.1. New Interrupt Preemption Configuration (`mipreemptcfg`) CSR

```
mipreemptcfg fields
 bits        description
 XLEN-1:4    (reserved)
 3:0         preemptmsk
```

`mipreemptcfg` is a WARL register used to configure the behavior of preemption. The highest legal value of `mipreemptcfg.preemptmsk` is IPRIOLEN.

A mask for filtering out the bits of a priority number that will participate in the selection for nested interrupt preemption is:

```
NIPPRIO_MASK = ~(2^(mipreemptcfg.preemptmsk) - 1)
```

5.1.2. New Interrupt enable threshold (`mithreshold`) CSR

A new WLRL M-mode CSR, `mithreshold`, determines the minimum interrupt priority (maximum priority number) for an interrupt to be considered enabled at the hart.

Register `mithreshold` implements exactly IPRIOLEN bits.

5.1.3. New Previous Interrupt Status (`mpistatus`) CSR

A new M-mode CSR, `mpistatus`, holds consolidated state of preempted context.

```
mpistatus
 Bits       Field            Description
```

```
 XLEN-1:30  (reserved)
 29:28      mpp[1:0]         Previous privilege mode, mirror of mstatus.mpp
 27         mpie             Previous interrupt enable, mirror of mstatus.mpie
 26:9       (reserved)
 8          psppush          Previous stack pointer push, mirror of msp.ppush,
if Smcspsw is implemented
 7:0        pithreshold[7:0] Previous interrupt enable threshold
```

5.2. Changed and new CSRs at Supervisor Level

```
       Number  Name         Description
 (NEW) 0x146   spistatus    Previous interrupt context
 (NEW) 0x1B1   sithreshold  Interrupt enable threshold
```

5.2.1. New Previous Interrupt Status (`spistatus`) CSR

```
spistatus
 Bits       Field            Description
 XLEN-1:29  (reserved)
 28         spp              Previous privilege mode, mirror of sstatus.spp
 27         spie             Previous interrupt enable, mirror of sstatus.spie
 26:9       (reserved)
 8          psppush          Previous stack pointer push, mirror of ssp.ppush,
if Sscspsw is implemented
 7:0        pithreshold[7:0] Previous interrupt enable threshold
```

The supervisor `sstatus` register has only a single `spp` bit (to indicate user/supervisor).

5.2.2. New Interrupt enable threshold (`sithreshold`) CSR

A new WLRL S-mode CSR, `sithreshold`, determines the minimum interrupt priority (maximum priority number) for an interrupt to be considered enabled at the hart.

Register `sithreshold` implements exactly IPRIOLEN bits.

5.3. Reset Behavior

In general in RISC-V, mandatory reset state is minimized but platform specifications or company policy might add additional reset requirements. Since the general privileged architecture states that mstatus.mie is reset to zero, interrupts will not be enabled coming out of reset.

> *For an S-mode execution environment, the EEI should specify that* `sstatus.sie` *is also reset on entry. It is then responsibility of the execution environment to ensure that is true before beginning execution in S-mode.*

5.3.1. Mandatory reset state

`mithreshold` resets to zero.

The reset behavior of other fields is platform-specific.

5.4. Operation

When `xithreshold` is a nonzero value, an interrupt sources *i* at with priority number `iprio[i]` is considered disabled irrespective of its interrupt-enable bit, when the following condition holds:

```
iprio[i] & NIPPRIO_MASK >= xithreshold
```

When `xithreshold` is zero, all enabled interrupt sources can contribute to signaling interrupts to the hart.

5.4.1. Trap behavior

The existing trap behavior is modified as follows:

When a trap is taken on interrupt *i* into level `x`, the current value of `xithreshold` is written to `xpistatus.pithreshold`. Additionally, `xithreshold` is set to `iprio[i]`. This automatically raises the interrupt enable threshold.

Synchronous exceptions do not modify the value of `xithreshold`.

5.4.2. Returns from Handlers

The behavior of an `xret` instruction is modified as follows:

`xret` sets `xithreshold` to `xpistatus.pithreshold`.

This automatically restores the interrupt enable threshold to the level of the previous context.

5.5. State Enable

If the Smstateen extension is implemented, then the bit 53 (ACLIC) in mstateen0 is implemented. If bit 53 (ACLIC) of a controlling mstateen0 CSR is zero, then access to the new CSRs `spistatus` and `sithreshold` by S-mode or a lower privilege mode results in an illegal instruction exception, except if the hypervisor extension is implemented, and the conditions for a virtual instruction exception apply, in which case a virtual instruction exception is raised when in VS or VU mode instead of an illegal instruction exception.

Chapter 6. Support for interrupt vector table (Smivt, Ssivt)

These extensions adds a new mode for taking interrupts via a dedicated interrupt vector table.

This increases the range of reachable addresses to the entire address space. This is useful when handlers are more than 2 MiB away from the vector table entry.

In this new mode, when an interrupt is taken, the hart hardware loads the vector table entry for the associated interrupt (table pointed to new `xivt` or `xeivt` CSR), masks off the least-significant bit (for IALIGN=16) or masks of the 2 least-significant bits (for IALIGN=32), and then jumps to the masked address.

6.1. Changed and new CSRs

```
       Number  Name         Description
       0x305   mtvec        M-mode Trap-handler base address / interrupt mode
 (NEW) 0x307   mivt         M-mode Interrupt-handler vector table base address
 (NEW) 0x308   meivt        M-mode External Interrupt-handler vector table base
address
       0x105   stvec        S-mode Trap-handler base address / interrupt mode
 (NEW) 0x107   sivt         S-mode Interrupt-handler vector table base address
 (NEW) 0x108   seivt        S-mode External Interrupt-handler vector table base
address
```

6.1.1. New `xivt` and `xeivt` CSR

Two new base addresses for vector tables are introduced. One for internal interrupt (major and local interrupts), and one for external interrupts.

Both `xivt` and `xeivt` are WARL XLEN-bit CSRs, which hold the base address of the interrupt vector table. The address held in `xeivt` is aligned on a 64-byte or greater power-of-two boundary. Values other than 0 in the low 6 bits are reserved of `xeivt` are reserved.

The actual alignment can be determined by writing ones to the low-order bits then reading them back.

The resulting vector tables, especially for external interrupts, can consume non-negligible memory space. To limit the overall memory footprint, `xivt` or `xeivt` values at different privilege levels may be configured to the same values, respectively.

6.1.2. New `xtvec` CSR Mode

Interrupt vector table mode is encoded as a new state in the existing `xtvec` WARL register, where `xtvec.mode` (the two least-significant bits) is `11`.

6.1.3. Smivt Changes to `xtvec` CSR Mode

The PC upon interrupt is changed as follows:

```
 mode  PC on Interrupt
 00    OBASE                                # Direct mode
 01    OBASE+4*exccode                      # Vectored mode
 11    M[VTBASE+XLEN/8*vtoffset] & VTMASK   # Interrupt vector table mode
 10    Reserved

where:
  M[a]     = Contents of memory address at address "a"
  VTMASK   = ~0x1 if IALIGN=16 or ~0x3 if IALIGN=32
  exccode  = xcause.Exception Code

  OBASE  = xtvec[XLEN-1:2]<<2   # Vector base is at least 4-byte aligned
  if external interrupt:
    VTBASE = xeivt[XLEN-1:6]<<6   # External Interrupt Vector Table base is at
least 64-byte aligned
    vtoffset = Minor Interrupt Identity
    # Corresponds to xtopei[26:16] with interrupt delivery from interrupt file
or ACLIC
  else
    VTBASE = xivt[XLEN-1:2]<<2    # Interrupt Vector Table base is at least 4-
byte aligned
    vtoffset = Major Interrupt Identity
```

In interrupt vector table mode, when interrupts are taken, the interrupt behavior is modified as follows: After executing the required side-effects as required with the existing behavior, the hart then fetches an XLEN-bit handler address with permissions corresponding to the handler's mode from the in-memory table whose base address (VTBASE). The trap handler function address is fetched from `VTBASE+XLEN/8*exccode`. If the fetch is successful, the hart clears the low bit(s) (depending on IALIGN) of the handler address, and sets the PC to this handler address. The masked vector table entry bit(s) are reserved and should be zero.

> *The original vectored mode simply jumps to an address in the trap vector table, while the interrupt vector table mode reads a handler function address from the table, and jumps to it in hardware.*

For permissions-checking purposes, the memory access to retrieve the function pointer for hardware vectoring is an *implicit* fetch at the privilege mode of the interrupt handler, and requires execute permission; read permission is irrelevant. It is recommended that the vector table fetches are ignored for hardware triggers and breakpoints.

Memory writes to the vector table require an instruction barrier (*fence.i*) to guarantee that they are visible to the instruction fetch.

It is recommended that the second fetch be ignored for hardware triggers and breakpoints.

6.1.4. Vector table fault handling

Faults that occur during the fetch of interrupt vector table entries are only recoverable, if the double trap extension(s) are implemented. For interrupts in M-mode, this additionally requires the Smrnmi extension.

> *Resuming after a fault on a vector table fetch is currently only seen as useful for instruction*

page faults.

In this case, if an exception trap occurs during the interrupt handler function address fetch, the following holds true: Depending on where the second trap is handled, the corresponding exception pc (`xepc` or `mnepc`) value is set to the exception pc of the first trap. In addition, `xtval` is either set to zero or written with the faulting address.

For simpler systems, we do not require that `xtval` is written with the faulting address. For systems with demand paging, `xtval` should be written with the faulting address to simplify page-fault handling code.

After returning from handling the synchronous exception, another trap will happen immediately. In case a higher priority interrupt became pending and enabled, this will trap first.

As long as synchronous exceptions can occur during the interrupt vector table fetch, side-affects of the trap may only be executed if they allow resuming of operation after the handling of the synchronous exception.

Resuming after a fault on a vector table fetch is currently only seen as useful for instruction page faults.

6.1.5. State Enable

If the Smstateen extension is implemented, then the bit 53 (ACLIC) in mstateen0 is implemented. If bit 53 (ACLIC) of a controlling mstateen0 CSR is zero, then access to the new CSR `sivt` by S-mode or a lower privilege mode results in an illegal instruction exception, except if the hypervisor extension is implemented, and the conditions for a virtual instruction exception apply, in which case a virtual instruction exception is raised when in VS or VU mode instead of an illegal instruction exception.

Chapter 7. Synchronous exceptions hardware vectoring- Smehv

The Smehv extension depends upon the Smivt extension.

Accelerating synchronous exception handlers through vectoring may reduce interrupt latency. During synchronous exceptions, interrupts are disabled. Acceleration of this critical segment decreases the time during which interrupts are masked. Typical examples of synchronous exceptions that occur as part of normal program execution include traps due to ECALLs, misaligned accesses, instruction page faults, or illegal instructions.

With this extension, synchronous exceptions are vectored using xtvec as an offset. The summarized behavior is the following:

```
mode  PC on Synchronous Exception
00       OBASE              # Direct mode
01       OBASE              # Vectored mode
11       OBASE+4*exccode    # Interrupt vector table mode
10       Reserved
```

Chapter 8. Trap handler push and pop extension (Smtp)

The trap handler push and pop extension adds instructions to accelerate trap handling, which additionally reduce code size. Trap handler latency and jitter is reduced by allowing the context save to happen speculatively in parallel to the vector table and handler fetch. Additional latency reduction is achieved by allowing late-preemption of the push sequence in favor of a higher priority interrupt or synchronous exception. Trap handling throughput is increased by allowing tail-chaing of interrupts or synchronous exceptions, skipping the context restore and save in between servicing two handlers. The instructions are designed to be applicable to all RISC-V trap handling schemes.

8.1. Trap handler push and pop functional overview

1. The tpush instruction
   - Conditionally performs a stack pointer swap by executing cspspush sp
   - Saves hart's state of the preempted context that can be overwritten in case of a nested preemption
   - Save registers x10-x15 (these are the caller saved registers available in RVC)
   - Adjusts the stack pointer to create the stack frame
   - Clear the ip bit if writable
   - Write trap information to argument registers
   - Optionally, enable interrupts and clear the xstatus.XDT flag if the double trap extension is implemented at the current privilege level

> *If a trap occurs during the sequence, then any operations which were executed before the trap may update architectural state and/or memory contents. The preempting trap is expected to fully save and restore the architectural state that it overwrites. If the trap occurs before updating the stack pointer, it needs to be assumed that the stacked context was overwritten. In this case, if the instruction is resumed, any store operations must be re-executed.*

The tpush instruction writes the value of the xcause CSR into register a0. This is beneficial also with vectored interrupts, as it allows sharing the same handler for similar interrupt sources. For example, if a peripheral exists multiple times in a system at different offsets, this can be handled with one handler.

The following table summarizes the written trap information:

Register	Synchronous Exception	Major Interrupt	External Interrupt (Interrupt Vector Table Mode)
a0	`xcause` (equal to `xcause.ExceptionCode`)	Major Identity	Minor Identity
a1	`xtval`	No Write	No Write
a2	`xtval2`	No Write	No Write
a3	`xtinst`	No Write	No Write

Software shall only use the information from the argument registers, as the corresponding CSRs may be overwritten by preempting traps. The values of xtval, xtval2, and xinst only need to be written to an argument register if the corresponding exception cause a write to the respective CSR.

As a consequence, the following prototype could serve for an interrupt handler at machine level.

```
void interrupt_handler(uint16_t id) __attribute__((interrupt, prestacked
("x10-x15,xepc,xpistatus")));
```

It is explicitly allowed for the push instruction to clobber any of the saved GPRs.

1. Second The tpopxret instruction
   - Disables interrupts
   - Set the xstatus.XDT flag if the double trap extension is implemented at the current privilege level
   - Restores registers x10-x15
   - Restores the hart's state to the point that the hart can return to the interrupted context
   - Adjust the stack pointer to destroy the stack frame
   - Conditionally performs a stack pointer swap by executing cspspop sp
   - Executes an xret according to the current level

   > *If a trap occurs during the sequence, then any operations which were executed before the trap may update architectural state. A trap may not occur after updating the stack pointer, if the context was not yet fully restored from the stack.*

   There is no specific order mandated for the context save/restore, and the individual parts may occur in parallel. Interrupts shall not be enabled before completing the context save/restore operations of the tpush instruction. Interrupts must be disabled before starting to restore context in the tpopxret instruction. As interrupts are disabled during the sequence as a side effect of the trap being taken, intermediate states during the execution of the instruction are not observable.

   If the Smtp extension is implemented, the first and last instructions of a trap handler must be a tpush and the last a tpopxret instruction, respectively.

   > *In contrast to the Zcmp push/pop instructions, Smtp instructions do not encode an incremental stack pointer adjustment in order to ease late preemption and tail chaining.*

   With these requirements, implementations are allowed to speculatively start executing an tpush instruction in case of a taken interrupt.

8.2. Late Preemption of the interrupt handler

While executing the tpush instruction, a hart may optionally check for higher priority pending and enabled interrupts at the current privilege mode, and switch to fetching this trap handler instead. If such an interrupt exists, the hart may abort executing the tpush instruction and trigger the following actions: - Update the `xcause.Exception Code` to the corresponding interrupt id. - Perform the trap handler fetch. - Start executing the trap handler, potentially skipping portions of tpush that have been executed by the aborted tpush.

There will be a point of no return during the tpush instruction. Specifically, once the interrupt pending bit is cleared, the tpush instruction must complete for the given interrupt source.

8.3. Tail chaining of interrupts handlers

While executing the tpopxret instruction, a hart may optionally check for any pending and enabled

interrupts at the current privilege mode. If such an interrupt exists, the hart may abort executing the tpopxret instruction and trigger the following actions: - Update the `xcause.Exception Code` to the corresponding interrupt id. - Perform the trap handler fetch. - Start executing the trap handler, while skipping the portions of tpush that have not been reverted by the aborted tpopxret.

Explicitly, `xepc`, `xpistatus` remain unchanged.

> *In the typical case of tail chaining, the check for another pending enabled interrupt is done after disabling interrupts at the start of the tpopxret instruction. In this case, only the conditional interrupt enablement of the tpush instruction would need to executed.*

Once the instruction retires, or the interrupts are disabled, no late preemption can occur.

8.4. Stack layout

To ease debugging, a stack layout is recommended for the saved context.

> *This stack layout is not mandated to allow e.g. shadow register sets*

```
* Smnip/Ssnip not present:

** XLEN = 32:

stack pointer adjustment: 32

24: x15
20: x14
16: x13
12: x12
8: x11
4: x10

** XLEN = 64:

stack pointer adjustment: 48

48: x15
40: x14
32: x13
24: x12
16: x11
8: x10

* Smnip/Ssnip present:

start of stackframe remains same. In addition:

** XLEN = 32

stack pointer adjustment: 32
```

```
32: xpistatus (XLEN)
28: xpepc (XLEN)

** XLEN = 32

stack pointer adjustment: 64

60: xpistatus (XLEN)
56: xpepc (XLEN)
```

8.5. Fault handling

Unless it can be guaranteed by design that faults causing synchronous exceptions cannot occur during the critical section, handling can only be resumed if the double trap extension is supported. The critical sections are defined as the time during execution of the tpush and tpopxret instruction, during which interrupts are disabled.

8.6. Debug

As the execution of tpush may start before the instruction is fetched, breakpoint conditions met might observe an inconsistent state. Therefore, if a breakpoint is triggered because of an tpush instruction, the breakpoint shall be handled as if it occurred on the subsequent instruction.

8.7. Instructions

8.7.1. tpush

Synopsis

Push trap context

Mnemonic

tpush preempt

Encoding

31						25	24	23				20	19				15	14			12	11				7	6					0
0	0	0	1	0	0	0	imm1	0	1	1	1		0	0	0	0	0	0	0	0		0	0	0	0	0	1	1	1	0	0	1	1
preempt													PRIV									SYSTEM										

Description

Saves the interrupted context at the beginning of a trap. If preempt is '1', interrupts will be enabled at the end of the sequence.

Behavior

The behavior in RISC-V assembly when preempt is '1' is as follows. If preempt is '0', enabling interrupts shall be skipped. The behavior is assuming Smnip is present. Otherwise, xpistatus will not be saved.

```
push:
  cspspush sp, sp           # Conditionally swap stack pointer
  addi sp, sp, -FRAMESIZE   # Adjust stack pointer
  sr a0, OFFSET(sp)         # Save a0
  sr a1, OFFSET(sp)         # Save a1
  sr a2, OFFSET(sp)         # Save a2
  sr a3, OFFSET(sp)         # Save a3
  sr a4, OFFSET(sp)         # Save a4
  sr a5, OFFSET(sp)         # Save a5
  sr a6, OFFSET(sp)         # Save a6
  csrr a0, xcause           # Get xcause of interrupted context to determine if
exception
  csrr a4, xepc             # Get xepc of interrupt context
  csrr a5, xpistatus        # Get xpistatus of interrupt context
  sr a4, OFFSET(sp)         # Save xepc
  sr a5, OFFSET(sp)         # Save xpistatus
  bgez a0, handle_exc       # Handle synchronous exception
  csrr a0, xtopi            # Read major interrupt number into a0
  li a1, xEI                # Write major identity of external interrupt to a1
  beq a0, a1, handle_ei     # Handle external interrupt
  # clear major interrupt pending bit - no single instruction equivalent
shift_idenity:
  srli a0, a0, 16           # Right shift topi value to have idenity aligned
with bit 0.
finish_push:
  csrrsi zero, xstatus, XIE # Enable interrupts
```

```
handle_ei:
  csrr a0, xtopei           # Read minor interrupt number into a0
  j shift_identity           # Jump back to rest of push sequence

handle_exc:
  csrr a1, xtval            # Get xcause of interrupted context
  csrr a2, xtval2           # Get xepc of interrupt context
  csrr a3, xtinst           # Get xpistatus of interrupt context
  j finish_push             # Jump back to rest of push sequence

where
  sr is sw on RV32 and sd on RV64
  lr is lr on RV32 and ld on RV64
```

Note that implementations do not need to execute these exact instructions, as long as the functional behavior is achieved.

While this assembly sequence clobbers a4 and a5 to save CSRs, this is not a requirement. SW shall not make assumptions about value of registers, with the exception of the a0-a3 as described earlier.

Included in: Smtp

8.7.2. tpopxret

Synopsis

Pop trap context

Mnemonic

tpopxret

Encoding

31–25	24–20	19–15	14–12	11–7	6–0
0000100	01000	00000	000	00000	1110011
			PRIV		SYSTEM

Description

Restores an interrupted context at the end of a trap.

Behavior

The behavior in RISC-V assembly is as follows. The behavior is assuming Smnip is present. Otherwise, the xpistatus CSR will not be restored.

```
pop:
  csrrsi zero, xstatus, XIE # Disable interrupts
  lr a4, OFFSET(sp)         # Restore xepc
  lr a5, OFFSET(sp)         # Restore xpistatus
  csrw xepc, a4             # Write xepc back to CSR
  csrw xpistatus, a5        # Write xpistatus back to CSR
  lr a0, OFFSET(sp)         # Restore a0
  lr a1, OFFSET(sp)         # Restore a1
  lr a2, OFFSET(sp)         # Restore a2
  lr a3, OFFSET(sp)         # Restore a3
  lr a4, OFFSET(sp)         # Restore a4
  lr a5, OFFSET(sp)         # Restore a5
  lr a6, OFFSET(sp)         # Restore a6
  addi sp, sp, FRAMESIZE    # Adjust stack pointer
  cspspop sp, sp            # Swap stack pointer with xsp
  xret

where
  sr is sw on RV32 and sd on RV64
  lr is lr on RV32 and ld on RV64
```

Included in: Smtp